

CONTENTS

- Overview
- 1 The orthodox path, and what it optimised for
- 2 The gap: quick decisions under uncertainty
- 3 Why the speed is architectural, not tuning
- 4 Typed in, typed out: the rules of engagement change
- 5 Primitives you compose, like logic gates
- 6 Calibration as the training objective
- 7 What you give up, and what it costs
- 8 Choosing: a portfolio, not a winner
- Glossary
- Check yourself
- Where to go next

System One Models: Typed Decisions Instead of Generated Text

Learn when a specialised model beats a general-purpose LLM, and exactly what you give up to get it.

For engineers who reach for a frontier LLM by default and want to know when that is the wrong tool · 22 September 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- What a large language model is, and that it generates text one token at a time
- Basic familiarity with JSON schemas or typed function signatures

Overview

Since ChatGPT, one shape of model has absorbed almost all the attention: an autoregressive language model, trained to be helpful to humans, that answers in free-form text. It was tuned for chat, then tuned again for coding agents. Both are open-ended tasks where a person reads the output.

Workflow automation is not that. Sorting ten thousand emails, routing a request to the right model, or checking whether a generated answer is safe are decisions, not compositions: the set of valid answers is known before the call, nobody reads the prose, and the same call happens millions of times. Running those through a model built to write sentences means paying for text generation you throw away, and waiting for tokens to come out one at a time.

TypeSafe AI's Jev, released in early access in September 2026, is the clearest public argument that this is a mismatch worth fixing in the model rather than around it. It takes unstructured state in and returns a typed value with a probability attached — no sentences at all. This lesson uses it as the concrete case for a more durable idea: the training objective and the decoding scheme, not the parameter count, decide which use cases a model can actually serve.

KEY TAKEAWAYS

- ✓ A System One model takes unstructured state and returns a schema-constrained value with a calibrated probability, instead of free-form text.
- ✓ Jev's speed is architectural rather than tuning: it samples every output in one pass, escaping the one-token-after-another floor of autoregressive decoding.
- ✓ TypeSafe's headline figures — 70–500 ms end to end, 40x–200x faster than frontier LLMs — are self-reported; the most grounded public number is Vercel's 18x at p95 on a live safety-review step.
- ✓ The model cannot hallucinate a fact into a sentence because it cannot produce a sentence: every output must fit a schema fixed before the call.
- ✓ RLHF optimises for human preference, RLVR for verifiable reward, RLCD for calibration — the objective a model was trained against is what limits where it fits.
- ✓ Giving up text is a real cost: no explanation, no chain of thought, and reported accuracy near mid-tier models rather than frontier ones.
- ✓ The practical move is a portfolio, not a winner: a generalist for open-ended work, a specialist for the high-volume decisions inside a workflow.

01 The orthodox path, and what it optimised for

0:00

Models have been tuned for the two use cases that dominated the application layer — chat, then coding agents — and the layers below reshaped themselves to serve them.

From 2022 onward, the target was a person in a chat window: a model that is helpful, harmless, and pleasant to read. When coding agents became mainstream, the target shifted to a second open-ended task — long, tool-using sessions where the model writes code and explains itself.

Both targets share a shape. The answer is unbounded text, a human or an agent reads it, and quality is judged by the whole response. Everything downstream was built for that shape: serving stacks tuned for token streaming, APIs that return strings, evaluation suites that score prose.

The video's framing is that the application layer exerts downward pressure on the stack, and the stack morphs to serve whatever the application layer rewards. That is efficient right up to the point where a use case with a different shape shows up and gets served badly by everything at once.

KEY POINTS

- Chat and coding agents are both open-ended tasks with a human reader at the end.
- The whole stack — serving, APIs, evals — was shaped around returning streamed text.
- A use case with a different shape inherits that shaping whether it suits it or not.

The shape of a chat answer versus a routing decision

"Explain this stack trace" has no bounded answer set and a person reads every word. "Which of these four queues should this ticket go to?" has exactly four valid answers and no reader at all — yet both are usually sent to the same model, over the same streaming API, and billed the same way.

02 The gap: quick decisions under uncertainty

0:40

Workflow automation stayed out of reach not because models were not smart enough, but because they were optimised against objectives that do not transfer to it.

TypeSafe's claim, as the video relays it, is that there is a schism in the orthodox line. Training a model on human preference (RLHF) produces answers people like. Training it further on verifiable reward (RLVR) produces answers that are checkably correct on problems with a right answer. Neither produces a probability you can act on when a decision has to be made now, under uncertainty, at volume.

The cost shows up as latency and money rather than as a wrong answer. A frontier model can sort your emails — it just takes seconds per item and pays for a paragraph of reasoning you discard. Multiply that by the volume automation implies and the use case dies economically long before it dies technically.

This is why the argument is about optimisation targets rather than intelligence. The video is explicit that neither path is superior in general; they are aimed at different things, and forcing one into the other's job is the mistake.

KEY POINTS

- RLHF optimises for what a person prefers to read.
- RLVR optimises for answers that can be checked against ground truth.
- Neither produces an actionable confidence number for a fast decision under uncertainty.
- Automation fails on unit economics and latency, not usually on capability.

The same task, two economics

Classifying one support email with a frontier model might take two seconds and a few hundred tokens of invisible reasoning. That is fine for one email and ruinous for 50,000 a day — the work is identical, the bill and the wall-clock time are not.

03 Why the speed is architectural, not tuning

2:04

An autoregressive model has a latency floor set by its decoding loop; parallel sampling removes the loop rather than making it faster.

An LLM produces token $N+1$ by conditioning on tokens $1..N$. That dependency is the whole trick, and it is also a serial chain: however fast your hardware, the tokens come out one after another until the model emits a stop.

Jev is non-autoregressive. All outputs for a request are computed in a single pass, so asking it ten questions costs roughly what asking it one costs — the questions do not queue behind each other. TypeSafe reports 70–500 ms end to end against 3–329 seconds for frontier LLMs on the queries it targets, and claims 40x–200x.

Treat those numbers as vendor figures, because they are. The most grounded public measurement so far is Vercel's CEO reporting 18x faster at p95 on a live safety-review step — a real production path rather than a benchmark, and an order of magnitude below the marketing number. That gap is the normal distance between a controlled comparison and your system, and it is still a large speedup.

KEY POINTS

- Autoregressive decoding is a serial chain: each token waits for the previous one.
- Parallel sampling computes every output in one pass, so more questions barely add latency.
- TypeSafe reports 70–500 ms versus 3–329 seconds for frontier LLMs, and 40x–200x overall.
- The best independent figure is 18x at p95 in production (Vercel), not 200x.
- A functional imitation is easy; matching the latency is what the architecture buys.

Ten questions for the price of one

Asking an autoregressive model to answer ten yes/no questions produces ten answers in sequence, and the tenth waits for the first nine. A parallel sampler evaluates all ten in the same pass, which is why batching questions into one Jev call is close to free while batching into one LLM call is not.

Where a 200x speedup is worth nothing

If the step is followed by a human reading the result, cutting 300 ms to 2 ms changes nothing anyone can feel. The speedup only converts into value when the step runs millions of times or sits on a critical path that a machine consumes.

You stop writing prompts and start declaring a schema; the model returns a value inside that schema with a probability attached.

With a chat model you send text and get text, and any structure you want — JSON, a category, a number — is something you ask for politely and then validate, because the model can always answer with an apology instead.

Jev inverts that. The possible outputs and their structure are declared before the call, and TypeSafe's claim is that the model "never makes type errors" and cannot hallucinate, for the blunt reason that it gives up string generation entirely. There is no channel through which a wrong-shaped answer could arrive. What comes back is a value plus a calibrated confidence.

This is the part that disorients people used to prompting. There is no room for "think step by step", no explanation to read, no way to ask why. You get a decision and a number, and the reasoning that would have been prose is now your problem to design around — usually by branching on the confidence rather than reading an argument.

KEY POINTS

- The answer space is declared up front, not requested in a prompt.
- Schema conformance is structural, so no output parser and no retry loop.
- It cannot hallucinate a fact into a sentence because it produces no sentences.
- You lose the explanation; the confidence number replaces it.
- Prompt-engineering intuitions mostly do not transfer.

The same guardrail, two shapes

With an LLM you ask for a verdict and hope: the answer may be JSON, may be JSON wrapped in prose, may be a refusal — so you validate, retry, and keep a fallback. With a typed decision model the verdict and its probability are the only things that can come back, so the failure mode moves from parsing to calibration.

```
// schematic – not TypeSafe's actual API
// declared before the call:
//   verdict: choice("allow", "review", "block")
// returned:
//   { verdict: "review", p: 0.81 }

if (p >= 0.95) autoApply(verdict);
else escalateToHuman();
```

05 Primitives you compose, like logic gates

4:48

The model offers a few decision shapes, and anything complicated is built by composing them in your own code.

The video describes three primitives: a categorical choice, a score over ordered options, and a yes/no probability. That is the entire vocabulary. It compares the feeling to being handed logic gates and registers again, with the abstractions still to be built on top.

That comparison is exact in a useful way. Expressiveness now lives in your program rather than in the prompt: a multi-step judgement becomes several typed calls whose results you combine, rather than one long instruction hoping the model holds the whole thing in mind.

It also means the ceiling is different. You cannot ask for something the primitives cannot express, and you cannot talk your way around a limitation. The design work moves from wording to decomposition — which of my decisions are actually categorical, and what do I do at each confidence level.

KEY POINTS

- Three shapes: a categorical choice, a score, and a yes/no probability.
- Complex behaviour comes from composing calls in code, not from a longer prompt.
- Expressiveness moves out of the prompt and into your program's structure.
- Anything the primitives cannot express is simply out of scope for the model.

Sorting a long list

The video's own demo is a long list of plants sorted in a second or two. Structurally it is one categorical decision repeated across every item, batched into a single pass — the kind of map-reduce over a dataset that is unremarkable to write and prohibitively slow to run through a chat model.

Composing two primitives

A moderation step can be a yes/no probability for "is this a policy violation" and, when that fires, a categorical choice of which policy — two cheap typed calls in sequence instead of one prompt asking for a JSON object with two fields and an explanation.

06 Calibration as the training objective

6:11

RLCD optimises the probabilities against outcomes rather than against what a person prefers, which is what makes the confidence number worth branching on.

TypeSafe calls its method Reinforcement Learning for Calibrated Decisions. The stated goal is that confidence tracks accuracy in aggregate: among the answers it returns at 0.9, roughly nine in ten should be right. A model optimised for human preference has no such guarantee — it learned to sound certain, and sounding certain is rewarded whether or not it is warranted.

Calibration is what turns a probability into a control signal. If the number is meaningful you can set a threshold, auto-apply above it, and route the rest to a person or to a bigger model — and you can tune that threshold against the cost of being wrong instead of guessing.

The method has not been published in detail, and the video is fair about that: the idea is not new, and it points at an open-source, roughly 421-million-parameter bidirectional BERT-style model built by someone on Reddit that runs on consumer hardware and does something similar in spirit. Narrow, specialised models predate the generative wave; what is new is the framing and the frontier-level intelligence claimed for one.

KEY POINTS

- RLCD optimises probabilities against outcomes, not against human preference.
- Calibration means 0.9 should be right about nine times in ten, in aggregate.
- A calibrated number is a control signal: threshold, auto-apply, escalate the rest.
- The method is not fully disclosed — treat the calibration claim as untested.
- Small encoder-style models have done narrow classification for years.

A threshold you can defend

With calibrated confidence, choosing 0.97 as the auto-apply cutoff is an argument about acceptable error rate: at that threshold roughly three in a hundred auto-applied decisions are wrong, and you decide whether that is tolerable. With an uncalibrated model the same number is superstition.

07 What you give up, and what it costs

5:31

No prose means no explanation, no chain of thought, and — on TypeSafe's own benchmark — accuracy closer to mid-tier models than to the frontier.

The trade is not free. On TypeSafe's own four-workflow benchmark the model lands near 68% accuracy, in the neighbourhood of mid-tier general models rather than the best ones. It is cheaper and faster by a wide margin, and less accurate than the frontier — which is exactly the trade you would expect and exactly the one the marketing figures do not lead with.

You also lose every affordance that comes from text. No reasoning to audit, no explanation to show a user, no ability to ask a follow-up in natural language, and no graceful handling of a case nobody declared in the schema. Debugging changes character: when it is wrong, you get a number, not an argument to poke at.

So the honest framing is a Pareto frontier rather than a replacement. On its shaped queries it competes with fast mid-tier models on cost and latency; on open-ended work it does not compete at all, because it cannot write a word.

KEY POINTS

- Reported accuracy is near mid-tier models, not frontier ones.
- No explanation, no chain of thought, nothing to audit when it errs.
- Anything outside the declared schema has no representation at all.
- Vendor benchmarks compare on the queries the vendor's model is shaped for.

Where it cannot go

"Draft a reply to this complaint" has no typed answer set — the output is the text. No amount of speed helps, because the thing being asked for is the very capability that was traded away.

08 Choosing: a portfolio, not a winner

6:53

Ask what shape the task is before asking which model is best — bounded answers at volume with no human reader is a different job from open-ended generation.

The durable lesson from the video is not about one model. It is that brute-forcing a general-purpose foundation model into every task was always a phase, and that as architectural and training-objective constraints bite, models optimised for particular shapes of work fill in the space around the generalists.

A usable test has three questions. Is the set of valid answers known before the call? Does a machine consume the result rather than a person? Does the call happen often enough that latency and unit cost matter? Three yeses describe a decision, and a decision is what a System One model is for. A single no — especially a human reader — usually points back at a general model.

And check your own numbers first. A 200x speedup on a step that takes 300 ms and runs a hundred times a day is a rounding error; the same speedup on a step in a hot path running millions of times is a different system. The claim that matters is the one measured on your workload, which is why Vercel's 18x at p95 is worth more than any benchmark table.

KEY POINTS

- Three yeses — bounded answers, machine consumer, high volume — mean it is a decision.
- A human reader at the end almost always argues for a general model.
- Measure your own p95 before believing any speedup multiple.
- Specialist and generalist are complements; the mistake is assuming one shape fits everything.

Applying the test to three calls

Routing a ticket to one of four queues: bounded, machine-consumed, high volume — a decision.

Summarising the ticket for an agent to read: unbounded and human-read — a general model.

Checking a generated answer against a safety policy before it ships: bounded, machine-consumed, on the hot path — a decision, and the case Vercel measured.

Glossary

System One model

TypeSafe's name for a model class that takes unstructured state and returns a typed, schema-constrained decision with a probability, rather than free-form text.

Autoregressive decoding

Generating output one token at a time, each conditioned on the tokens before it — the reason a language model's response time grows with its length.

Parallel sampling

Computing all of a request's outputs in a single pass instead of in sequence, so the number of questions barely affects latency.

Calibration

The property that a model's stated confidence matches its real accuracy: answers given at 0.9 confidence are right about 90% of the time.

RLCD

Reinforcement Learning for Calibrated Decisions — TypeSafe's training method, which optimises the model's probabilities against outcomes rather than human preference.

RLHF

Reinforcement Learning from Human Feedback — training a model to produce the answers human raters prefer.

RLVR

Reinforcement Learning with Verifiable Rewards — training against answers that can be checked automatically, such as a test suite passing.

Schema-constrained output

Output whose shape and allowed values are fixed before the call, so a malformed or out-of-range answer cannot be produced.

p95 latency

The response time that 95% of requests come in under — a truer picture of a production path than an average.

Pareto frontier

The set of options where improving one dimension, such as latency, requires giving up another, such as generality.

Check yourself

Answer first, then open each one.

Why can't a frontier LLM simply be tuned to match Jev's latency?

Because the limit is structural, not a matter of tuning: an autoregressive model emits tokens in sequence, each conditioned on the last, so a response takes as long as its length. Parallel sampling removes the sequence rather than speeding it up.

The claim is that Jev 'cannot hallucinate'. What exactly does that mean, and what does it not mean?

It means it cannot produce a wrong-shaped or invented output, because it generates no free text — every answer must be a value from a schema fixed before the call. It does not mean the answer is right: it can confidently pick the wrong option from that schema.

What makes a calibrated probability more useful than an LLM saying it is confident?

Calibration ties the number to observed accuracy, so 0.9 means roughly nine in ten. That makes it a control signal you can threshold against the cost of being wrong. A preference-trained model learned that sounding certain is rewarded, so its confidence carries no such guarantee.

A step in your pipeline takes 400 ms and runs 200 times a day, and a vendor offers a 200x speedup. What is wrong with taking that at face value?

Two things. The multiple comes from the vendor's own shaped benchmark — the best independent figure is 18x at p95 in production — and at that volume even a real 200x saves under two minutes a day, so the speedup buys nothing regardless of whether it is true.

Your task is to write a customer-facing apology email. Why is a System One model the wrong tool, no matter how fast it is?

The deliverable is the text itself, and generating text is precisely the capability traded away for speed and type safety. The answer set is unbounded and a person reads it — two of the three signs that the job belongs to a general model.

Where to go next

- Take the highest-volume model call in your system and write down every answer it is allowed to return; if you can enumerate them, it is a decision wearing a generation's clothes.
- Measure that call's current p95 latency and daily volume before evaluating any speedup claim — the multiple only matters against your own numbers.
- Try the same classification with a fine-tuned encoder-style model of a few hundred million parameters: it may be cheaper still and run on hardware you already own.
- Design one confidence threshold end to end — pick the cutoff, decide what happens below it, and work out what an error above it costs you.
- Read TypeSafe's System One announcement alongside a sceptical write-up, and note which figures are self-reported and which were measured by someone else.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.